

- 1) Write a C program to simulate
 - a. Bankers' algorithm for the purpose of deadlock avoidance

```
#include <stdio.h>

int main()
{
    int n, m, i, j, k;

    // Input number of processes and resources
    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int allocation[n][m], max[n][m], need[n][m];
    int available[m];

    // Input Allocation Matrix
    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }

    // Input Maximum Demand Matrix
    printf("Enter Maximum Demand Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            scanf("%d", &max[i][j]);
        }
    }

    // Input Available Resources
    printf("Enter Available Resources:\n");
    for(i = 0; i < m; i++)
    {
        scanf("%d", &available[i]);
    }
}
```

```
}
```

```
// Calculate Need Matrix
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    for(j = 0; j < m; j++)
```

```
    {
```

```
        need[i][j] = max[i][j] - allocation[i][j];
```

```
    }
```

```
}
```

```
int finish[n], safeSequence[n];
```

```
int work[m];
```

```
// Initialize finish array
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
    finish[i] = 0;
```

```
}
```

```
// Initialize work = available
```

```
for(i = 0; i < m; i++)
```

```
{
```

```
    work[i] = available[i];
```

```
}
```

```
int count = 0;
```

```
// Safety Algorithm
```

```
while(count < n)
```

```
{
```

```
    int found = 0;
```

```
    for(i = 0; i < n; i++)
```

```
    {
```

```
        if(finish[i] == 0)
```

```
        {
```

```
            int possible = 1;
```

```
            // Check if need <= work
```

```
            for(j = 0; j < m; j++)
```

```
            {
```

```
                if(need[i][j] > work[j])
```

```
                {
```

```

        possible = 0;
        break;
    }
}

// If process can execute
if(possible)
{
    for(k = 0; k < m; k++)
    {
        work[k] += allocation[i][k];
    }

    safeSequence[count] = i;
    count++;

    finish[i] = 1;
    found = 1;
}
}

// If no process found
if(found == 0)
{
    printf("\nSystem is not in a safe state.\n");
    return 0;
}

// Safe state output
printf("\nSystem is in a safe state.\n");
printf("Safe sequence is: ");

for(i = 0; i < n; i++)
{
    printf("P%d", safeSequence[i]);

    if(i != n - 1)
    {
        printf(" -> ");
    }
}

```

```

printf("\n");

return 0;
}

```

```

C:\Users\admin\Desktop\safe
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2

Process returned 0 (0x0)   execution time : 64.936 s
Press any key to continue.

```

2) Deadlock

```
#include <stdio.h>
```

```

int main()
{
    int n, r, i, j, k;
    printf("Enter the number of processes: ");
    scanf("%d", &n);

```

```
printf("Enter the number of resources: ");
scanf("%d", &r);
```

```
int allocation[n][r], request[n][r];
int available[r];
int finish[n], safeSequence[n];
```

```
printf("Enter the allocation matrix:\n");
```

```
for(i = 0; i < n; i++)
{
    printf("Process %d: ", i);

    for(j = 0; j < r; j++)
    {
        scanf("%d", &allocation[i][j]);
    }
}
```

```
printf("Enter the request matrix:\n");
```

```
for(i = 0; i < n; i++)
{
    printf("Process %d: ", i);

    for(j = 0; j < r; j++)
    {
        scanf("%d", &request[i][j]);
    }
}
```

```
printf("Enter the available resources: ");
```

```
for(i = 0; i < r; i++)
{
    scanf("%d", &available[i]);
}
```

```
for(i = 0; i < n; i++)
```

```

{
    finish[i] = 0;
}

int count = 0;

while(count < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int possible = 1;

            for(j = 0; j < r; j++)
            {
                if(request[i][j] > available[j])
                {
                    possible = 0;
                    break;
                }
            }
            if(possible)
            {
                for(k = 0; k < r; k++)
                {
                    available[k] += allocation[i][k];
                }

                safeSequence[count] = i;
                count++;

                finish[i] = 1;
                found = 1;
            }
        }
    }
    if(found == 0)
    {
        printf("\nSystem is not in safe state.\n");
        return 0;
    }
}

```

```
}

printf("\nSystem is in safe state.\n");

printf("Safe Sequence is: ");

for(i = 0; i < n; i++)
{
    printf("P%d ", safeSequence[i]);
}

printf("\n");

return 0;
}
```

```
C:\Users\admin\Desktop\neh: X + v
Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter the request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter the available resources: 0 0 0

System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1

Process returned 0 (0x0)   execution time : 70.810 s
Press any key to continue.
```